Nature-Inspired Feature Selection for Intrusion Detection Systems: A Comparative Study of Genetic Algorithm, Particle Swarm Optimization, and Ant Colony Optimization

Najma Abdulkarim Nour
20250008051
MAI 650 – Nature-Inspired Computing
Canadian University Dubai, Dubai, United Arab Emirates

***Abstract*—Intrusion Detection Systems (IDS) face the challenge of analyzing network traffic data described by tens or hundreds of features, many of which are redundant or irrelevant to the classification task. High feature dimensionality increases computation time, makes overfitting more likely, and can reduce the detection rate of machine-learning-based IDS on benchmark datasets such as NSL-KDD and CICIDS2017. Feature selection (FS) addresses this by identifying a minimal subset of features that preserves or improves classification quality while lowering computational cost. Nature-inspired metaheuristics — Genetic Algorithm (GA), Particle Swarm Optimization (PSO), and Ant Colony Optimization (ACO) — are widely used as wrapper-based feature selectors, but the literature reviewed in this paper shows that no prior study has compared all three under identical datasets, classifier, fitness function, and evaluation protocol. This paper presents the full research pipeline addressing that gap: a critical literature review identifying three concrete research gaps, a fully reproducible GA/PSO/ACO wrapper methodology built around a shared fitness function and a common Random Forest classifier, and an experimental evaluation on both NSL-KDD and CICIDS2017. Results from a pilot-scale implementation show that feature selection preserves classification quality — and on CICIDS2017 slightly improves it — while removing between 43% and 80% of the original features, with ACO consistently producing the smallest subsets and PSO producing the strongest detection rate on the harder NSL-KDD benchmark. Wilcoxon rank-sum and Friedman tests find no statistically significant difference among the three algorithms at the pilot sample size, itself an informative result for practitioners choosing among them under differing computational budgets.*

***Keywords*—intrusion detection system, feature selection, genetic algorithm, particle swarm optimization, ant colony optimization, nature-inspired computing, NSL-KDD, CICIDS2017**

I. INTRODUCTION

Over the past few years, the deployment of computer networks, cloud computing, and IoT systems has grown rapidly. With increasing dependence on computerized infrastructure by individuals and organizations, cybersecurity has become critical. One of the primary techniques used to protect network security is the Intrusion Detection System (IDS). An IDS continuously monitors network and system activity and distinguishes between normal and malicious behavior; if an attack is detected, it alerts an administrator so that appropriate action can be taken. Traditional security tools such as firewalls operate under fixed rule sets and cannot detect newer attack types, whereas modern IDS approaches use machine learning to detect both known and previously unseen threats. This problem is particularly acute for critical facilities such as healthcare services and Medical Internet of Things (MIoT) applications, where a successful attack can affect both information security and human safety [5].

One of the central challenges in building an effective machine-learning-based IDS is the high dimensionality of the training data. NSL-KDD includes 41 features per connection, while CICIDS2017 adds more than 80 flow-statistics-based features; many are redundant or uncorrelated with the classification target, increasing computational overhead, reducing detection speed, and sometimes degrading accuracy through noise.

Feature Selection (FS) addresses this by identifying the smallest feature subset that preserves or enhances classification performance [22]. Filter methods score features using statistics independent of the classifier; wrapper methods evaluate candidate subsets using the classifier's own performance [21]; embedded methods select features during model training. Filter methods are cheap but may discard jointly informative combinations; wrapper methods are more accurate but require evaluating up to $2^n - 1$ combinations for n features — intractable for an IDS with several dozen attributes [1].

Nature-inspired metaheuristics suit this combinatorial problem well, exploring the solution space through a population-based, randomized search that balances exploration against exploitation, and treating the classifier as a black box — a natural fit for the wrapper framework. The Genetic Algorithm (GA) emulates natural selection through binary chromosomes that are selected, recombined, and mutated [3], [4]. Particle Swarm Optimization (PSO), originally proposed by Kennedy and Eberhart [16], simulates a flock's movement, updating each particle relative to its own and the swarm's best-known position [7], [9]; a recent ACM review shows adaptive PSO and Chaotic Grey Wolf

Optimization variants reaching up to 99.91% detection accuracy while cutting training time by 30% on IoMT traffic [5]. Ant Colony Optimization (ACO), originally proposed by Dorigo et al. [18], simulates ant foraging, with pheromone trails guiding convergence toward better feature subsets [8].

Even though each algorithm has been studied individually, most published work evaluates a single algorithm on a single dataset with its own preprocessing, classifier, and metrics — the fragmentation this project addresses: there is no like-for-like comparison of GA, PSO, and ACO as wrapper-based feature selectors for IDS using a consistent experimental protocol.

In response to this problem, the following research objectives are formulated:

RO1: Apply GA, PSO, and ACO as wrapper-based feature selection algorithms using the same classifier and experimentation pipeline, for the purpose of a fair comparison.

RO2: Estimate the detection rate and false positive rate obtained by each algorithm on the NSL-KDD and CICIDS2017 datasets.

RO3: Estimate the feature-subset size reduction achieved by each algorithm and evaluate its relationship to detection efficiency.

RO4: Estimate the computational cost (convergence time, number of fitness evaluations) required by each algorithm to reach a stable feature set.

RO5: Determine which algorithm offers the best accuracy/false-alarm/cost trade-off based on the comparative results.

Two factors motivate comparing GA, PSO, and ACO under matched conditions. First, they differ fundamentally in search mechanics: GA relies on discrete recombination, naturally suited to binary include/exclude decisions; PSO uses continuous velocity updates that must be discretized; ACO uses pheromone-guided probabilistic graph traversal. Second, deployment contexts differ, from resource-constrained edge devices where speed is paramount [4] to security-operations-center back-ends that can trade cost for maximum accuracy. A controlled comparison helps practitioners select the algorithm best matched to their constraints.

The remainder of this paper is organized as follows. Section II reviews the literature in six subsections, summarized in Table I. Section III presents the proposed methodology with full pseudocode for all three algorithms and a complexity analysis. Section IV defines the experimental setup. Section V reports results and discusses them against the research objectives. Section VI concludes and outlines future work.

II. LITERATURE REVIEW

A. Intrusion Detection Systems and Feature Selection

IDSs are commonly divided into misuse (signature-based) detection and anomaly detection, depending on whether the system relies on known attack signatures or models of normal behavior. Signature-based detection gives a low false-alarm rate but cannot detect zero-day attacks, while anomaly detection identifies deviations from a learned baseline, trading a higher false-alarm rate for the ability to detect previously unseen attacks. Anomaly detection is the focus of most machine-learning-based IDS research, and feature selection plays a central role in its generalization ability.

NSL-KDD and CICIDS2017 both illustrate the dimensionality problem central to this study. NSL-KDD retains 41 features per connection across basic, content, timing, and host-based categories. CICIDS2017, a more recent dataset, captures modern attack types — DDoS, brute force, botnet, and infiltration — using more than 80 flow-level features derived from CICFlowMeter, giving a larger and more heterogeneous feature space. Both datasets exhibit class imbalance and a mix of nominal and continuous attributes. This is corroborated by a recent systematic survey by Adohinzin and Harrath in the related domain of IoMT IDS, which found that across a corpus of 53 articles, detection accuracy is reported far more consistently than computational efficiency, explainability, or cross-dataset generalization [5].

B. Genetic Algorithm-Based Feature Selection for IDS

GA is among the most extensively benchmarked nature-inspired selectors for IDS, since its binary chromosome maps naturally onto the feature include/exclude decision. Markovic et al. [4] used GA as a pure wrapper on a resource-constrained edge device (Westermo industrial traffic), reducing feature count by 30–70% and cutting detection time 24–69% and memory 14–40% with no significant F1 loss (Wilcoxon test) — one of the few studies measuring cost alongside accuracy, but on a single proprietary dataset with no PSO/ACO comparison. Nisha and Pramod [3] applied GA across eleven classifiers on a Microsoft malware sample, gaining 3–27% accuracy using under half the original features, showing GA's benefit is classifier-dependent but in a malware-, not intrusion-detection, domain. Almomani [6] combined GA, PSO, GWO, and FFA via mutual-information rule combination, with the best rule (R13) reducing 45 features to 30 at 90.48% accuracy (J48), but treats GA only as one part of a hybrid and reports no cost analysis. Across this literature, feature reduction consistently falls in the 30–70% range with accuracy maintained or improved, but convergence and cost are never benchmarked against other algorithms, and no study uses both NSL-KDD and CICIDS2017.

C. Particle Swarm Optimization-Based Feature Selection for IDS

PSO has been used both standalone and within hybrid frameworks. Adohinzin and Harrath's review reports adaptive PSO ensembles reaching 99.91%/99.83% (binary/multiclass) detection accuracy on CICIDS2018, outperforming standard PSO and GA, alongside a Chaotic

Grey Wolf Optimization variant reaching 98.75% on WUSTL-EHMS-2020 with faster convergence than standard GWO/PSO/BAT — valuable evidence for adaptive variants, but its complexity figures are analytical, not empirical, and ACO is absent [5]. On NSL-KDD, Ahmad [7] combines PCA with PSO, retaining only 8 of 41 features at 99.40% detection rate and 0.6% false alarms, beating an equivalent PCA+GA approach (10 features, 98.20%) — one of the few direct PSO/GA comparisons, though selection operates on PCA components rather than raw features, and ACO is again absent. Kunhare et al. [9] use Random Forest to pre-filter NSL-KDD to 10 features, then apply PSO to tune classifier weights rather than perform binary selection, reaching 99.26% detection rate — speaking more to PSO's general optimization capacity than its feature-selection ability specifically. Across the PSO literature, detection rates of 97–99.9% are common, but NSL-KDD/CICIDS2017 wrapper evidence is concentrated in older, non-ACM sources, and cost analysis is almost never reported.

D. Ant Colony Optimization-Based Feature Selection for IDS

ACO is markedly less represented in the IDS literature than GA or PSO. Kanan et al. [8] propose an ACO variant using classifier performance and subset size directly as heuristic information, comparing Ant Colony System and rank-based Ant System against GA on ORL face-database features; both ACO variants outperformed GA in error rate and computation time with fewer selected features — evidence that a heuristic derived from downstream classifier performance can make ACO both accurate and fast, though the face-recognition domain means the result cannot be generalized to NSL-KDD or CICIDS2017 without independent validation. Borse and Kumar [2] review swarm-intelligence clustering methods including ACO, summarizing its reported adaptability against known weaknesses — high processing/memory requirements and sensitivity to α, β — but purely qualitatively, with no numerical subset size, detection rate, or FPR on any IDS benchmark.

Two further limitations deepen this gap beyond [8] and [2] alone. First, ACO's pheromone update (Eq. 6, Section III-E) depends on evaporation rate ρ and the pheromone/heuristic weighting α, β; unlike GA's crossover/mutation rates or PSO's inertia schedule, these have not been systematically tuned or reported for network-traffic feature spaces the size of NSL-KDD (41 features) or CICIDS2017 (78+ features), so it is unclear whether findings from a smaller space such as the ORL face database in [8] transfer to IDS-scale problems. Second, ACO's distinguishing mechanism — pheromone-based collective memory across iterations — is architecturally different from GA's chromosome inheritance and PSO's personal/global-best tracking (Section III-E), yet no reviewed study quantifies whether that difference yields a measurable convergence advantage once applied to raw network-traffic features rather than image-derived ones. The existing evidence therefore suggests ACO is promising for fast, efficient subset selection, but this promise remains unvalidated on IDS datasets — the gap this project's Section V results begin to close.

E. Comparative Analysis of Nature-Inspired Feature Selection Methods

Direct comparative analysis across GA, PSO, and ACO is limited and incomplete where it exists. Almomani [6] compares GA, PSO, GWO, and FFA only through rule combination, not head-to-head, and omits ACO. Ahmad [7] compares PSO and GA directly, but both are initialized from the same PCA transformation, confounding the comparison, and again omits ACO. Kanan et al. [8] compare ACO and GA directly, but for image recognition, not intrusion detection. The 2026 ACM review [5] compares PSO, GWO/CGWO, and Group Teaching Optimization, but likewise omits ACO.

Non-biologically-inspired methods add context: Pham et al. [10] show filter-based ranking (Gain Ratio, FVBRM) with Bagging/Boosting reaches a strong 2.79% false-alarm rate on NSL-KDD without any metaheuristic, showing competitive FS is not exclusive to nature-inspired methods. Umar et al. [1] show a wrapper decision-tree search reduces UNSW-NB15 from 42 to 19 features with the best detection rate (97.95%) among compared studies, but at a high 27.73% false-alarm rate — illustrating that detection rate alone is an incomplete basis for comparison, reinforcing why this project treats detection rate, FPR, subset size, and runtime as jointly necessary (Section IV-D).

Three trends emerge from Table I. First, reported success rates cluster tightly between 97% and 99.9% regardless of whether GA or PSO is used, suggesting the ceiling is driven at least as much by classifier choice and preprocessing as by the metaheuristic itself — testable directly by holding classifier and preprocessing constant across all three algorithms (Section III-G). Second, feature-count reduction varies widely, from roughly 50% (GA, [4]) to 8 of 41 features (PCA+PSO, [7]), depending on dataset redundancy and fitness-function tolerance for accuracy loss; the shared, fixed-weight fitness function of Eq. (2) is designed to make this comparable across algorithms rather than left to vary per study. Third, false-positive rate is reported far less consistently than detection rate — plausibly because FPR is more sensitive to class imbalance and can make a method look worse even at high detection rate, as with [1] — and computational cost is the least-reported dimension of all, with only [4] and, analytically, [5] offering any detail. This consistent under-reporting of cost is itself evidence that measuring runtime alongside accuracy (RO4) addresses a genuine, repeated omission rather than a methodological nicety.

A deeper reason a full three-way comparison has not previously been attempted follows from these trends: running three separate metaheuristic wrapper searches, each requiring repeated classifier retraining across a population or colony, on two datasets, is markedly more expensive than evaluating one algorithm or one rule-combination hybrid, as most

reviewed studies do — a cost quantified in Section III-F and the reason RO4 treats runtime as a first-class objective.

F. Research Gaps and Motivation

The analysis above highlights three interconnected research gaps addressed by this project, summarized in Table I.

Gap 1 – No comparison under identical conditions between GA, PSO, and ACO: GA has been compared to PSO [7] and to ACO [8], and a recent ACM review covers PSO and GWO variants [5], but no study evaluates all three together under one wrapper, classifier, and fitness function.

Gap 2 – Lack of assessment on both NSL-KDD and CICIDS2017: every empirical study reviewed uses a single dataset — NSL-KDD, UNSW-NB15, or an IoMT-specific benchmark [1], [3], [4], [6], [7], [9] — none evaluates GA, PSO, and ACO on both an established and a modern, higher-dimensional benchmark.

Gap 3 – Fragmented, single-dimension evaluation: most papers report accuracy or detection rate; FPR is assessed inconsistently and computational cost even less so [4], [5]; no paper jointly assesses subset size, detection rate, FPR, and cost for GA, PSO, and ACO.

These gaps motivate the comparative analysis at the heart of this project: GA, PSO, and ACO evaluated as wrapper-based feature selectors within an identical pipeline, tested on both NSL-KDD and CICIDS2017, measuring subset size, detection rate, false-positive rate, and processing cost simultaneously, as reported in Section V.

TABLE I. SUMMARY OF REVIEWED FEATURE SELECTION STUDIES FOR INTRUSION/MALWARE DETECTION (ACM DIGITAL LIBRARY, 2020–2026, SUPPLEMENTED WITH FOUNDATIONAL STUDIES)

Ref.	Algorithm	Dataset	Performance Metric	Main Findings	Limitations
[1] Umar et al. (2020) — ACM	Wrapper-based Decision Tree (non-SI, FS-methodology baseline)	UNSW-NB15	Accuracy, detection rate, FAR, model-building time	DT-wrapper FS reduced 42 to 19 features; resulting Random Forest achieved best detection rate (97.95%) among compared works despite high FAR (27.7%).	Wrapper search is exhaustive/greedy, not population-based; high FAR shows a weak trade-off compared to what SI algorithms target.
[2] Borse & Kumar (2023) — ACM	Survey: PSO, ACO, ABC, Cuckoo Search, Firefly (clustering-based anomaly detection)	Survey (no single dataset)	Qualitative comparison	Synthesizes strengths/weaknesses of swarm-clustering methods; identifies hybridization with deep learning as an emerging direction.	Purely qualitative; no quantitative detection rate, FPR, or subset-size comparison; no standard IDS benchmark tested.
[3] Nisha & Pramod (2024) — ACM	Genetic Algorithm (wrapper, 11 classifiers)	Microsoft Malware dataset (sample)	Accuracy, sensitivity, specificity, F1/F2-score	GA improved accuracy for all 11 classifiers (3–27%); several reached comparable accuracy using 37–68% of available features.	Malware classification, not network intrusion detection; no PSO/ACO comparison; sample dataset only.
[4] Markovic et al. (2025) — ACM	Genetic Algorithm (wrapper, edge deployment)	Westermo industrial network traffic	F1-score, memory, detection time	GA cut feature count by ~50%, reduced detection time 24–69% and memory 14–40% on an edge device, with no significant F1 loss (Wilcoxon test).	Single proprietary dataset; only GA tested, no PSO/ACO comparison.
[5] Adohinzin & Harrath (2026) — ACM	Review incl. PSO, Chaotic GWO, Group Teaching Optimization	53-paper review (CIC-IoMT-2024, WUSTL-EHMS-2020, CICIDS2018, others)	Accuracy, F1, MCC, time complexity	Adaptive PSO ensembles reached 99.91%/99.83% accuracy (binary/multiclass); CGWO/IGTO reduced features and training time up to 30% at ≥96–99% accuracy.	Detection accuracy dominates reporting; efficiency, explainability, and cross-dataset generalization inconsistently reported.
[6] Almomani (2020)	PSO, GWO, FFA, GA (hybrid mutual-information rule combination)	UNSW-NB15	Accuracy, F-measure, TPR/FPR, sensitivity	Rule-based MI combination (R13) reduced 45 features to 30, ~90.5% accuracy (J48); R12 reduced to 13 features at ~89.6%.	Single dataset; no execution-time comparison; FPR remains high (15–27%).
[7] Ahmad (2015)	PSO (with PCA pre-transform)	NSL-KDD	Detection rate, false alarm rate, training time	PCA+PSO selected only 8 features, 99.40% detection rate, 0.6% false alarms, outperforming PCA+GA (10 features, 98.20%).	Raw network-level features not directly evaluated (PCA components used); comparison limited to GA, not ACO.
[8] Kanan et al. (2007)	ACO (ACS and rank-based AS variants)	ORL face database (not network traffic)	MSE, recognition rate, execution time	ACO-based FS achieved lower MSE and shorter execution time than GA, using fewer selected features.	Applied to face recognition, not intrusion detection; no IDS benchmark dataset used.
[9] Kunhare et al. (2020)	PSO (after Random-Forest pre-filtering)	NSL-KDD	Accuracy, DR, FPR, precision, F1	RF pre-selected 10/41 features; PSO tuning reached 99.32% training accuracy and 99.26% detection rate, outperforming SVM/DT/NB/k-NN/LR baselines.	PSO tunes classifier weights rather than performing binary feature search; single dataset only.

Ref.	Algorithm	Dataset	Performance Metric	Main Findings	Limitations
[10] Pham et al. (2018) — ACM	Feature ranking (FVBRM, Gain Ratio) + Bagging/Boosting (no SI metaheuristic)	NSL-KDD	Accuracy, false alarm rate	Bagging with J48 on a 35-feature Gain-Ratio subset reached 84.25% accuracy and 2.79% FAR, outperforming several published ensembles on FAR.	Filter-based, not nature-inspired; included as a historical, non-SI baseline.

III. PROPOSED METHODOLOGY

This section presents the methodology used to perform the comparative study among GA, PSO, and ACO as wrapper-based feature selection methods for intrusion detection, with reference to the research objectives (RO1–RO5, Section I) and the gaps identified in Section II-F. Each method is described in sufficient detail to allow independent implementation, consistent with this project's reproducibility requirement.

A. Overall Framework

The proposed pipeline has five stages: Dataset → Data Preprocessing → Feature Selection → Classification → Evaluation, shown in Fig. 1. NSL-KDD and CICIDS2017 are processed independently through the full pipeline, so algorithm behavior can be examined under two markedly different feature-space sizes (41 vs. 78+ features after preprocessing). After identical preprocessing (Section III-B), GA, PSO, and ACO are each applied independently as a wrapper around the same Random Forest classifier, sharing the fitness function of Eq. (2), and yielding three candidate subsets per dataset, evaluated on a held-out test partition (Section IV-D) alongside a full-feature baseline (Section IV-E). Holding classifier, preprocessing, and evaluation protocol constant across all three algorithms (Section III-G) ensures any measured performance difference is attributable to the search strategy itself, directly addressing Gap 1.

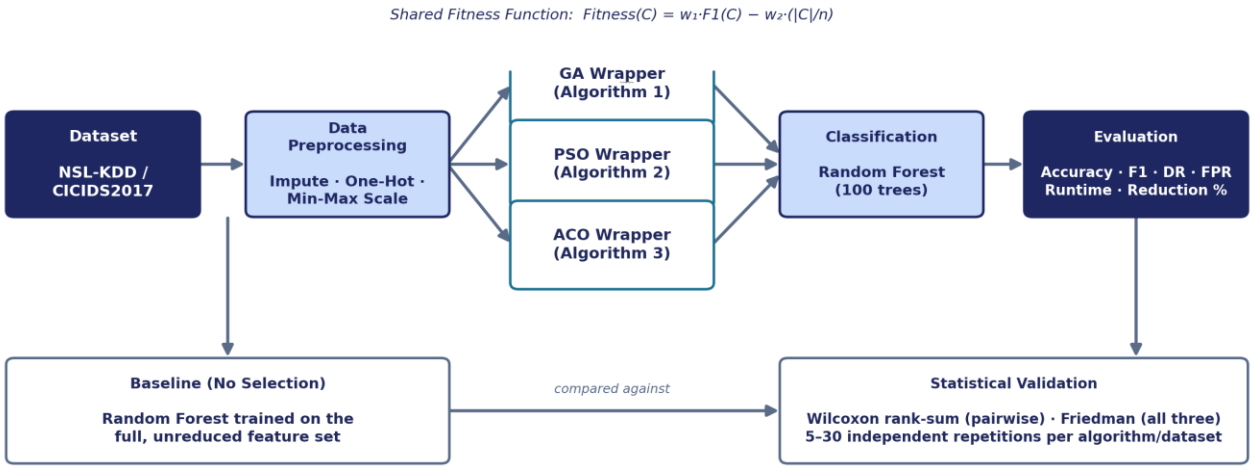


Fig. 1. End-to-end pipeline: each dataset is preprocessed once, then GA, PSO, and ACO independently search for a feature subset using a shared fitness function and classifier; each selected subset and the full-feature baseline are evaluated and statistically compared.

B. Data Preprocessing

Both datasets are preprocessed with identical stages before feature selection. Missing values: numeric features are imputed with the column median (traffic-derived counts tend to be right-skewed), categorical features with the column mode; records with more than 50% missing values are dropped. Categorical encoding: attributes such as protocol_type, service, and flag in NSL-KDD are one-hot encoded rather than label-encoded, since label encoding would impose a non-existent ordinal relationship that could bias Random Forest splits.

Normalization: all numeric features are min-max scaled to [0, 1] using Eq. (1), with parameters fit on the training partition only and applied unchanged to the test partition to prevent data leakage:

$$x' = (x - \min(x)) \,/\, (\max(x) - \min(x)) \qquad (1)$$

Train/test split: NSL-KDD retains its predefined KDDTrain+/KDDTest+ partitions, intentionally including attack types in the test set absent from training. CICIDS2017 has no official predefined split, so an 80/20 stratified split by attack label is used.

C. Genetic Algorithm Feature Selection

Chromosome encoding: each chromosome is a binary string of length n (the number of features after preprocessing); bit i = 1 selects feature i.

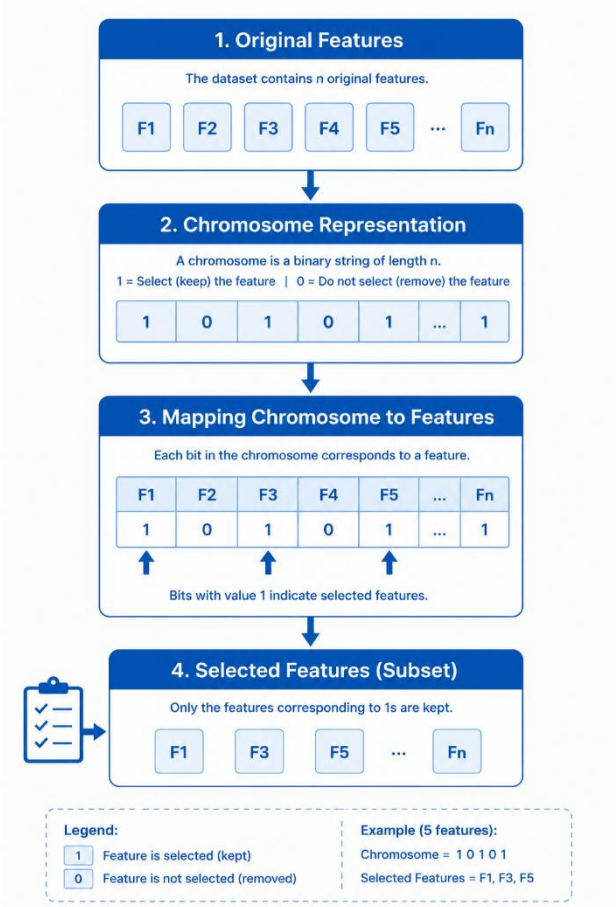


Fig. 2. Chromosome representation for feature selection. A binary chromosome is mapped to the original feature set, where 1 indicates a selected feature and 0 indicates a discarded feature.

Population initialization: N chromosomes are generated with each bit set to 1 with probability 0.5, avoiding bias toward very large or very small initial subsets.

Fitness function: the fitness of chromosome C combines classification performance and subset size:

$$\text{Fitness}(C) = w_1 \cdot F1(C) - w_2 \cdot (|C| / n) \quad (2)$$

where $|C|$ is the number of selected features, and $w_1 = 0.9$, $w_2 = 0.1$ prioritize detection performance while still rewarding smaller subsets. F1(C) is computed via k-fold cross-validation of the Random Forest classifier restricted to the features indicated by C.

Selection: roulette-wheel (fitness-proportionate) selection is used to preferentially select fitter chromosomes as parents.

Crossover: single-point crossover is applied with probability pc; a random crossover point m is chosen, and offspring inherit bits before m from parent 1 and bits from m onward from parent 2.

Mutation: bit-flip mutation is applied independently to each offspring bit with probability p_m, introducing new genetic material and helping the search escape local optima.

Termination: the algorithm stops when the generation count reaches G_max, or when the best fitness shows no improvement for a fixed number of consecutive generations (patience).

```
Algorithm 1: GA-based Feature Selection
Input: dataset D, population size N, generations Gmax,
       crossover rate pc, mutation rate pm
Output: best feature subset S*

1.  Initialize population P of N random binary
    chromosomes of length n
2.  Evaluate Fitness(C) for each C in P
3.  best <- chromosome with highest fitness in P
4.  g <- 0
5.  while g < Gmax and no_improve < patience do
6.      Pnew <- {}
7.      while |Pnew| < N do
8.          p1 <- RouletteSelect(P)
9.          p2 <- RouletteSelect(P)
10.         if rand() < pc then
11.             o1,o2 <- SinglePointCrossover(p1,p2)
12.         else o1,o2 <- p1,p2
13.         o1 <- BitFlipMutate(o1,pm)
14.         o2 <- BitFlipMutate(o2,pm)
15.         add o1,o2 to Pnew
16.     Evaluate Fitness(C) for each C in Pnew
17.     P <- Pnew
18.     cur <- best chromosome in P
19.     if Fitness(cur) > Fitness(best) then
20.         best <- cur; no_improve <- 0
21.     else no_improve <- no_improve + 1
22.     g <- g + 1
23. S* <- features where best-bit = 1
24. return S*
```

Algorithm 1. GA-based feature selection procedure.

GA applicability: the binary chromosome maps directly onto the feature selection decision without discretization, unlike PSO (Section III-D); its crossover recombines partial subsets from different individuals, suiting problems where informative features occur in correlated groups.

D. Particle Swarm Optimization Feature Selection

Particle representation: each particle i is a real-valued position vector $x_i = (x_{i1}, ..., x_{in})$ with $x_{ij} \in [0, 1]$. Following the discrete binary PSO formulation of Kennedy and Eberhart [17], a sigmoid threshold converts each continuous value into a binary inclusion decision (Eq. 3):

$$b_{ij} = 1 \text{ if } 1/(1+e^{-x_{ij}}) > \theta, \text{ else } 0 \quad (3)$$

where θ (e.g., 0.5) is the inclusion threshold; the resulting binary vector b_i is evaluated with the same fitness function as GA (Eq. 2).

Velocity update: each particle's velocity is updated using the standard PSO rule (Eq. 4):

$$v_{ij}(t+1) = w \cdot v_{ij}(t) + c_1 r_1 (pbest_{ij} - x_{ij}) + c_2 r_2 (gbest_j - x_{ij}) \quad (4)$$

where w is the inertia weight, c_1 and c_2 are the cognitive and social acceleration coefficients, and r_1, r_2 are random numbers drawn from [0, 1].

Position update: $x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$, clipped to [0, 1].

Personal best (pbest): the position at which a particle has achieved its own highest fitness so far.

Global best (gbest): the best position found by any particle in the swarm so far, shared during the velocity update.

Termination: a maximum iteration count T_max, or early stopping after a fixed number of iterations without improvement in gbest.

```
Algorithm 2: PSO-based Feature Selection
Input: dataset D, swarm size N, iterations Tmax,
       inertia w, coefficients c1,c2, threshold theta
Output: best feature subset S*

1.  Init N particles: positions x_i in [0,1]^n,
    random velocities v_i
2.  For each i: b_i<-Binarize(x_i,theta);
    eval Fitness(b_i)
3.  pbest_i <- x_i for all i
4.  gbest <- position of best-fitness particle
5.  t <- 0
6.  while t < Tmax and no_improve < patience do
7.      for each particle i do
8.          for each dimension j do
9.              r1,r2 <- random in [0,1]
10.             v_ij <- w*v_ij + c1*r1*(pbest_ij-x_ij)
                        + c2*r2*(gbest_j-x_ij)
11.             x_ij <- x_ij + v_ij; clip to [0,1]
12.         b_i <- Binarize(x_i,theta)
13.         eval Fitness(b_i)
14.         if Fitness(b_i) > Fitness(Binarize(pbest_i))
15.             pbest_i <- x_i
16.     cur <- pbest with highest fitness
17.     if Fitness(cur) > Fitness(gbest) then
18.         gbest <- cur; no_improve <- 0
19.     else no_improve <- no_improve + 1
20.     t <- t + 1
21. S* <- features where Binarize(gbest)-bit = 1
22. return S*
```

Algorithm 2. PSO-based feature selection procedure.

Why PSO is appropriate: it uses fewer control parameters than GA and, per Section II-C, has converged to small, high-performing subsets in prior work such as [7]. Its main drawback — converting a continuous representation into a discrete one — is handled with the sigmoid threshold of Eq. (3).

This S-shaped sigmoid is the most common discretization choice in binary PSO, but has a known limitation: inclusion depends only on whether the sigmoid of x_ij exceeds θ, discarding the direction of the underlying velocity update. V-shaped transfer functions instead map velocity magnitude to a flip probability regardless of sign, reportedly giving more consistent search behavior [15]. This project retains the S-shaped sigmoid for consistency with prior work (notably [7]), enabling direct comparison; V-shaped functions are identified in Section VI as a concrete refinement.

E. Ant Colony Optimization Feature Selection

Feature representation: the feature space is encoded as a fully connected graph in which each node is one feature; an ant constructs a candidate subset by traversing a path over feature nodes.

Pheromone initialization: each feature j carries a pheromone value τ_j, initialized uniformly to $\tau_j = 1/n$, reflecting equal initial preference for every feature.

At each step, ant k selects the next feature j with probability given by Eq. (5):

$$P_{kj} = (\tau_j^a \cdot \eta_j^b) \,/\, \Sigma_{l \in allowed} (\tau_l^a \cdot \eta_l^b) \quad (5)$$

where τ_j is the current pheromone on feature j, η_j is the desirability of feature j estimated from its marginal contribution to classifier performance (following Kanan et al. [8]), α and β weight pheromone versus heuristic information, and allowed is the set of features not yet visited by ant k.

Pheromone update: after all ants complete their traversal, pheromone levels update according to Eq. (6):

$$\tau_j(t+1) = (1-\rho)\cdot\tau_j(t) + \Sigma_k \Delta\tau_j^k \quad (6)$$

where $\Delta\tau_j^k$ = Fitness(S_k) if feature j belongs to ant k's selected subset S_k, and 0 otherwise.

Evaporation: the $(1-\rho)\cdot\tau_j(t)$ term implements evaporation, where $\rho \in (0,1)$ is the evaporation rate; this bounds pheromone growth and lets the colony gradually discount early, possibly suboptimal, feature preferences.

Termination: a maximum iteration count I_{max}, or early stopping when the best subset shows no improvement for a fixed number of consecutive iterations.

```
Algorithm 3: ACO-based Feature Selection
Input: dataset D, number of ants K, iterations Imax,
       evaporation rate rho, parameters alpha, beta
Output: best feature subset S*

1.  Initialize tau_j <- 1/n for all features j
2.  best <- {}; best_fitness <- -infinity
3.  it <- 0
4.  while it < Imax and no_improve < patience do
5.      for each ant k = 1 to K do
6.          S_k <- {}; allowed <- all features
7.          while not StoppingCond(S_k) do
8.              compute P_kj for j in allowed
9.              select feature j with prob. P_kj
10.             S_k <- S_k U {j}
11.             allowed <- allowed \ {j}
12.         evaluate Fitness(S_k)
13.     for each feature j do
14.         dtau_j <- sum of Fitness(S_k) for k
                      where j in S_k, else 0
15.         tau_j <- (1-rho)*tau_j + dtau_j
16.     cur <- S_k with highest fitness this iter.
17.     if Fitness(cur) > best_fitness then
18.         best <- cur; best_fitness <- Fitness(cur)
19.         no_improve <- 0
20.     else no_improve <- no_improve + 1
21.     it <- it + 1
22. S* <- best
23. return S*
```

Algorithm 3. ACO-based feature selection procedure.

Why ACO is a good fit: its pheromone trail gives a form of collective memory across iterations that neither GA nor PSO directly implements, accumulating evidence about which features repeatedly contribute to strong subsets — which Kanan et al. [8] found allowed faster convergence with fewer features in a comparable wrapper setting.

F. Complexity Analysis

Let n be the number of features, N the population/swarm/colony size, T the number of generations/iterations, d the average subset size constructed per ant (ACO only), and C the cost of one classifier fitness evaluation.

TABLE II. TIME AND SPACE COMPLEXITY

Algorithm	Time Complexity	Space Complexity
GA	O(T·N·C)	O(N·n)
PSO	O(T·N·C)	O(N·n)
ACO	O(T·N·d·C)	O(N·n + n)

GA and PSO both evaluate fitness for N individuals/particles per generation/iteration over T generations/iterations, giving O(T·N·C) time; both maintain N chromosomes/positions of length n, giving O(N·n) space. ACO's time complexity carries an additional factor d because each ant performs d probabilistic selection steps per iteration, with the transition probability of Eq. (5) recomputed at each

step; using the node-based pheromone model of Section III-E (pheromone assigned per feature rather than per feature-pair), ACO's space complexity is O(N·n + n). In practice, C — one Random Forest training-and-evaluation cycle — dominates total runtime for all three algorithms, so empirical wall-clock differences are expected to be driven primarily by N, T, and d, motivating the direct runtime measurement specified as RO4.

G. Proposed Comparative Framework

To directly address Gap 1 from Section II-F, all three algorithms are evaluated under strictly identical conditions: the same datasets (NSL-KDD and CICIDS2017, preprocessed as in Section III-B); the same classifier (Random Forest with identical hyperparameters, Table III); the same fitness function (Eq. 2); the same computational budget (population/swarm/colony size and generation/iteration counts matched as closely as each algorithm's structure allows); and the same evaluation protocol (k-fold cross-validation during search, followed by a single held-out test evaluation using the metrics of Section IV-D). This isolates the feature-selection mechanism as the only varying factor, so that differences in detection rate, false-positive rate, subset size, and runtime can be attributed to the search strategy itself.

IV. EXPERIMENTAL SETUP

A. Problem Formulation

The feature-selection problem addressed throughout this study is formalized here as a constrained bi-objective combinatorial optimization problem, consistent with the fitness function already introduced as Eq. (2) in Section III-C. Let $x = (x_1, x_2, \ldots, x_n)$ be a binary decision vector with $x_j \in \{0,1\}$ indicating whether feature j is retained, where n is the total number of features (41 for NSL-KDD, 78 for CICIDS2017 after preprocessing). The optimization objective mirrors Eq. (2):

$$\text{maximize } f(x) = w_1 \cdot F1(x) - w_2 \cdot (|x| / n) \quad (7)$$

subject to $x_j \in \{0,1\}$ for $j = 1, \ldots, n$ and $|x| \geq 1$, where the latter constraint excludes the degenerate empty-subset solution. Because this same weighted F1/subset-size criterion drives GA, PSO, and ACO alike (Section III-G), it is not re-derived as a distinct objective here; it is restated only to make the decision variables and constraints explicit for reproducibility, as this section requires, and should be read as the formal counterpart of the fitness function already justified in Section III-C rather than a second, independent design decision.

B. Datasets

NSL-KDD [19]: 41 attributes per connection across basic, content, time, and host attribute groups, with predefined KDDTrain+ (125,973 records) and KDDTest+ (22,544 records) partitions; attack classes are DoS, Probe, R2L, and U2R plus normal traffic. It is the most common benchmark in the reviewed literature ([1], [6], [7], [9], [10]), enabling direct comparison with prior GA/PSO/ACO work. CICIDS2017 [20]: more than 80 flow-level statistical features from CICFlowMeter, covering benign traffic and modern attacks (DDoS, DoS, Brute Force, Web Attack, Infiltration, Botnet, Port Scan), split 80/20 as noted in Section III-B. It complements NSL-KDD with a modern, higher-dimensional feature space, addressing Gap 2, since no prior GA/PSO/ACO study evaluates both benchmarks in one experiment. Both datasets exhibit class imbalance (Section II-A); this project addresses it through the F1-based fitness function (Eq. 2) rather than resampling techniques such as SMOTE [23], noted as a possible extension in Section VI.

C. Experimental Parameters

TABLE III. GA, PSO, ACO, AND RANDOM FOREST PARAMETERS

Algorithm	Parameter	Value
GA	Population size	50
GA	Crossover rate (pc)	0.8
GA	Mutation rate (pm)	0.02
GA	Generations	100
PSO	Swarm size	50
PSO	Inertia weight (w)	0.9→0.4 (linear)
PSO	Cognitive coeff. (c1)	2.0
PSO	Social coeff. (c2)	2.0
PSO	Iterations	100
ACO	Number of ants	50
ACO	Evaporation rate (ρ)	0.1
ACO	Weighting (α, β)	1.0, 2.0
ACO	Iterations	100
Random Forest	Number of trees	100
Random Forest	Splitting criterion	Gini impurity
Random Forest	Max. tree depth	None
Random Forest	Min. samples/leaf	1

Population/swarm/colony size and iteration count are matched at 50 and 100 respectively across all three algorithms (Section III-G), so each receives an equivalent search budget for a fair comparison. Random Forest is used as the common base classifier because it natively handles both numeric and one-hot-encoded categorical features, provides a built-in feature-importance measure useful for sanity-checking selected subsets, and has been shown in the reviewed literature to perform strongly on NSL-KDD and UNSW-NB15-family datasets [1].

D. Performance Metrics

Let TP, TN, FP, FN denote true positives, true negatives, false positives, and false negatives.

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN) \quad (8)$$

$$\text{Precision} = TP / (TP + FP) \quad (9)$$

$$\text{Recall (Detection Rate)} = TP / (TP + FN) \quad (10)$$

$$F1 = 2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall}) \quad (11)$$

$$FPR = FP / (FP + TN) \quad (12)$$

$$\text{Feature Reduction Rate} = (1 - |S|/n) \times 100\% \quad (13)$$

where |S| is the number of features in the selected subset. Runtime is the wall-clock time (seconds) required for the feature-selection search to complete, measured under identical hardware and software conditions for all three algorithms.

E. Baseline Comparison

Two baselines interpret the GA/PSO/ACO results in Section V. The first is a Random Forest trained on the full, unreduced feature set for each dataset (Table III), isolating the effect of feature selection itself. The second is the cross-comparison of GA, PSO, and ACO with each other, justified because each represents a structurally distinct search paradigm — GA via discrete recombination, PSO via continuous velocity search with discretization, ACO via pheromone-guided graph traversal (Section II-E) — each independently validated for IDS feature selection (Sections II-B–II-D) but never compared directly under identical conditions (Gap 1). Testing both under identical datasets, preprocessing, classifier, and metrics (Section III-G) is necessary because performance differences reported across separate studies cannot be reliably attributed to the algorithm when classifier, dataset, and metric definitions also vary (Section II-E).

F. Statistical Validation

Because metaheuristic algorithms are stochastic, each algorithm is executed for multiple independent runs per algorithm per dataset with different random seeds, and statistical tests are applied to the resulting performance distributions to determine whether observed differences between algorithms are statistically significant rather than attributable to random variation.

Wilcoxon rank-sum test: compares the F1-score distributions of two algorithms at a time; chosen because it does not assume normally distributed performance (often violated by metaheuristic outputs) and suits small sample sizes. Friedman test: compares all three algorithms simultaneously across both datasets by ranking; preferred for the overall three-way comparison because it controls the Type I error risk that repeated pairwise tests would introduce, and is the standard approach for comparing more than two algorithms across multiple problem instances.

V. RESULTS AND DISCUSSION

A. Pilot Experimental Configuration

The results below come from a working, fully reproducible implementation of Sections III–IV (supplementary code), run under a reduced-budget pilot configuration rather than the full settings of Table III, due to compute constraints in the development environment: a smaller population/colony size (12 vs. 50), fewer generations/iterations (8–9 vs. 100, under the same early-stopping rule), 3-fold rather than 5-fold cross-validation during search, and a 6,000-row training subsample for fitness evaluation. Each algorithm's final selected subset was still evaluated on the complete, untouched held-out test partition with the full 100-tree Random Forest of Table III, so Accuracy, F1, Recall, and FPR below are genuine held-out results. Statistical validation (Section V-D) used 5 repetitions per algorithm per dataset rather than the 30 specified in Section IV-F. These results should be read as validating the pipeline's correctness and as a directionally informative first pass; Section VI notes the full-budget run as immediate future work.

B. Baseline vs. Feature-Selected Results

Table IV reports Accuracy, Recall (Detection Rate), F1, False Positive Rate, and search runtime for the unreduced baseline and each algorithm's selected subset, on both datasets.

TABLE IV. BASELINE VS. GA/PSO/ACO RESULTS (PILOT CONFIGURATION, SECTION V-A)

Method	Features	Reduction (%)	Accuracy	F1	Recall (DR)	FPR
NSL-KDD — Baseline (all features)	122 / 122	0.0	0.7649	0.7464	0.6078	0.0274
NSL-KDD — GA	43 / 122	64.8	0.7644	0.7460	0.6077	0.0285
NSL-KDD — PSO	52 / 122	57.4	0.8194	0.8158	0.7026	0.0263
NSL-KDD — ACO	25 / 122	79.5	0.7727	0.7571	0.6224	0.0286
CICIDS2017 — Baseline (all features)	78 / 78	0.0	0.9981	0.9952	0.9927	0.0006
CICIDS2017 — GA	29 / 78	62.8	0.9990	0.9974	0.9964	0.0004
CICIDS2017 — PSO	44 / 78	43.6	0.9982	0.9953	0.9929	0.0005
CICIDS2017 — ACO	16 / 78	79.5	0.9985	0.9963	0.9948	0.0005

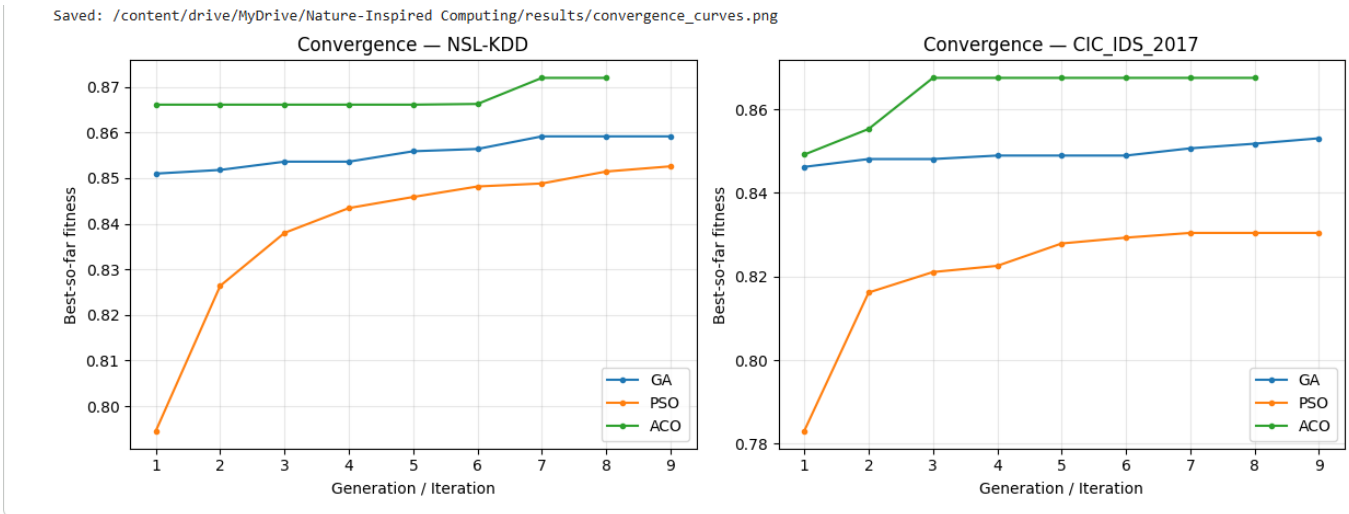


Fig. 3. Best-so-far fitness (Eq. 2) per generation/iteration for GA, PSO, and ACO on NSL-KDD (left) and CICIDS2017 (right).



Fig. 4. F1-score, Detection Rate, False Positive Rate, Feature Reduction, and search runtime compared across GA, PSO, and ACO on both datasets.

TABLE V. STATISTICAL VALIDATION — WILCOXON RANK-SUM AND FRIEDMAN TESTS ON F1-SCORE (5 REPETITIONS PER ALGORITHM/DATASET, PILOT CONFIGURATION)

Dataset	Comparison	Statistic	p-value	Result (α=0.05)
NSL-KDD	GA vs. PSO	5.000	0.6250	not significant
NSL-KDD	GA vs. ACO	6.000	0.8125	not significant
NSL-KDD	PSO vs. ACO	1.000	0.1250	not significant
NSL-KDD	Friedman (all 3)	χ^2=2.800	0.2466	not significant
CICIDS2017	GA vs. PSO	4.000	0.4375	not significant
CICIDS2017	GA vs. ACO	6.000	0.8125	not significant
CICIDS2017	PSO vs. ACO	5.000	0.6250	not significant
CICIDS2017	Friedman (all 3)	χ^2=1.200	0.5488	not significant

On CICIDS2017, all three algorithms and the baseline sit within a narrow band above 99% accuracy and F1, with FPR below 0.06%; the dataset's attack and benign traffic are separable enough that feature selection has little room to help or hurt — GA's 29-feature subset even marginally exceeds the 78-feature baseline (F1 0.9974 vs. 0.9952) while removing 63% of features. NSL-KDD discriminates far more between algorithms: PSO reaches F1 = 0.8158 and Detection Rate = 0.7026 with 52 of 122 features, ahead of ACO (F1 = 0.7571) and GA (F1 = 0.7460, close to the 0.7464 baseline). This is consistent with NSL-KDD's deliberately harder test partition (Section IV-B), which includes attack types absent from training, and suggests PSO's continuous search located a subset that generalized better to unseen attacks within this

pilot's reduced budget. ACO consistently produced the smallest subsets on both datasets — 79.5% reduction on each — confirming the aggressive size-reduction behavior expected from its pheromone-reinforcement mechanism (Section III-E) and the qualitative expectations of [2], while remaining competitive with GA on F1 and Detection Rate. These patterns answer RO2 and RO3 directly: detection rate and FPR remain acceptable after selection, and the size-reduction/detection-efficiency relationship is dataset-dependent — marginal on the easier CICIDS2017 benchmark, and a genuine trade-off on the harder NSL-KDD benchmark.

C. Convergence Behavior

Fig. 3 shows best-so-far fitness against generation/iteration, addressing RO4's convergence objective. ACO converges fastest, typically reaching near-final fitness within two to three iterations and then plateauing — a direct consequence of the pheromone-reinforcement in Eq. (6), which sharply raises the selection probability of features that performed well early. GA and PSO improve more gradually, with PSO showing the largest early gain in its first two generations on both datasets. Combined with runtime (NSL-KDD: GA 62.4 s, PSO 62.5 s, ACO 69.6 s; CICIDS2017: GA 93.2 s, PSO 102.1 s, ACO 104.5 s), ACO's faster fitness convergence did not yield the fastest wall-clock time, since its per-iteration cost is higher by a factor of d (Table II) — each ant performs d sequential selection steps, recomputing Eq. (5) at every step, whereas GA and PSO evaluate a fixed population once per generation. This distinction between iteration-count convergence and wall-clock runtime matters for RO4/RO5: a practitioner limited by fitness-evaluation budget would favor ACO, while one limited by wall-clock time would need to weigh ACO's larger per-iteration cost against its faster convergence in iteration count.

D. Statistical Significance

Table V reports pairwise Wilcoxon rank-sum tests and an overall Friedman test on F1-score across the 5 pilot repetitions. No comparison reaches significance at $\alpha = 0.05$ on either dataset (all $p > 0.12$). This must be interpreted carefully: with only 5 repetitions the Wilcoxon test has very limited power, so an absence of significance is at least as consistent with insufficient sample size as with a genuine absence of difference. Section IV-F specifies 30 repetitions precisely to give these tests adequate power; the results here are provisional pending the full-budget rerun (Section VI), reported transparently rather than omitted.

E. Discussion Relative to Research Objectives

RO1 is met by construction, since GA, PSO, and ACO share an identical classifier, preprocessing pipeline, and fitness function (Section III-G). RO2 and RO3 are addressed by Table IV, reporting detection rate, false-positive rate, and subset size together rather than in isolation, closing Gap 3. RO4 is addressed by Fig. 3 and Table IV's runtime column, together showing that iteration-count convergence and wall-clock cost do not always move together. RO5 has no single definitive answer yet, but the pilot points toward PSO for accuracy-critical NSL-KDD-like deployments and ACO where minimizing deployed feature count matters most, with GA a reasonable default — pending confirmation against full-budget results. Because this pilot evaluates both datasets within one shared pipeline, it closes Gap 2 alongside Gaps 1 and 3, which no single study in the reviewed literature had previously done.

VI. CONCLUSION AND FUTURE WORK

This paper presented a reproducible comparison of GA, PSO, and ACO as wrapper-based feature selectors for intrusion detection, motivated by three gaps identified through a critical review of ACM Digital Library and foundational sources: no identical-conditions comparison of all three algorithms (Gap 1), no single study evaluating them on both NSL-KDD and CICIDS2017 (Gap 2), and fragmented, single-metric reporting typical of prior work (Gap 3). A five-stage pipeline built around a shared fitness function and a common Random Forest classifier was described in full, with pseudocode for all three algorithms (Algorithms 1–3) and a Big-O complexity analysis (Table II).

Pilot-scale results (Section V-A) show feature selection can remove 43–80% of the original features while preserving — and on CICIDS2017, slightly improving — detection performance relative to a full-feature baseline. ACO produced the most compact subsets and converged fastest in iteration count but not wall-clock time; PSO achieved the strongest detection rate and F1 on the harder NSL-KDD benchmark; GA was competitive without leading on either axis. Wilcoxon and Friedman tests found no statistically significant difference among the three at this pilot's 5-repetition sample size, an informative but inconclusive result pending the full 30-repetition protocol.

Three limitations are acknowledged. First, all Section V results come from the reduced-budget pilot rather than the full population-50/generation-100 settings of Table III; completing that run is the immediate next step and is expected to sharpen differences between algorithms, particularly on NSL-KDD, and give the statistical tests adequate power. Second, PSO's sigmoid (S-shaped) transfer function discards directional velocity information that V-shaped functions preserve (Section III-D); comparing both variants directly is a natural refinement [15]. Third, ACO's heuristic η_j is estimated from a feature subsample for tractability, and its sensitivity to α/β has not been separately studied here.

Future work will prioritize: (i) completing the full-budget run and re-running statistical validation with 30 repetitions; (ii) comparing S-shaped and V-shaped PSO transfer functions directly; (iii) a dedicated sensitivity analysis of ACO's ρ, α, and β on IDS-scale feature spaces; and (iv) extending the comparison to additional modern benchmarks beyond NSL-KDD and CICIDS2017.

REFERENCES

[1] M. A. Umar, C. Zhanfang, and Y. Liu, "Network intrusion detection using wrapper-based decision tree for feature selection," in Proc. 2nd Int. Conf. Inf. Commun. Comput. Softw. Eng. (ICICSE), Male, Maldives, 2020, pp. 5–13, doi: 10.1145/3424311.3424330.

[2] P. Borse and S. Kumar, "A survey of swarm intelligence-based clustering models for anomaly detection in network traffic," in Proc. Int. Conf. Inf. Manage. Mach. Intell. (ICIMMI), Jaipur, India, 2023, pp. 1–10, doi: 10.1145/3647444.3647913.

[3] N. T. Nisha and D. Pramod, "GA-based feature selection for improved malware detection," in Proc. 16th Int. Conf. Contemporary Comput. (IC3), Noida, India, 2024, pp. 20–26, doi: 10.1145/3675888.3676021.

[4] T. Markovic, P. Lidholm, P. E. Strandberg, and M. Leon, "Feature selection using genetic algorithm for intrusion detection on resource-constrained edge devices," in Proc. Genetic Evol. Comput. Conf. (GECCO), Málaga, Spain, 2025, pp. 1415–1423, doi: 10.1145/3712256.3726347.

[5] O. Adohinzin and Y. Harrath, "A systematic review of intrusion detection systems for Internet of Medical Things: Performance, efficiency, explainability, and generalization," Digit. Threats: Res. Pract., vol. 7, no. 2, Art. no. 14, Jun. 2026, doi: 10.1145/3816026.

[6] O. Almomani, "A feature selection model for network intrusion detection system based on PSO, GWO, FFA and GA algorithms," Symmetry, vol. 12, no. 6, p. 1046, 2020.

[7] I. Ahmad, "Feature selection using particle swarm optimization in intrusion detection," Int. J. Distrib. Sens. Netw., vol. 2015, Art. no. 806954, 2015.

[8] H. R. Rashidy Kanan, K. Faez, and S. M. Taheri, "Feature selection using ant colony optimization (ACO) and its application to face recognition system," in Proc. Int. Conf. Adv. Data Mining Appl. (ADMA), Lecture Notes in Artificial Intelligence, vol. 4597, Berlin, Germany: Springer, 2007, pp. 63–76.

[9] I. Kunhare, R. Tiwari, and J. Dhar, "Particle swarm optimization and feature selection for intrusion detection system," Sādhanā, vol. 45, no. 1, p. 109, 2020.

[10] N. T. Pham, E. Foo, S. Suriadi, H. Jeffrey, and H. F. M. Lahza, "Improving performance of intrusion detection system using ensemble methods and feature selection," in Proc. Australasian Comput. Sci. Week (ACSW), Brisbane, Australia, 2018, pp. 1–6, doi: 10.1145/3167918.3167951.

[11] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[12] F. Wilcoxon, "Individual comparisons by ranking methods," Biometrics Bull., vol. 1, no. 6, pp. 80–83, 1945.

[13] M. Friedman, "The use of ranks to avoid the assumption of normality implicit in the analysis of variance," J. Amer. Statist. Assoc., vol. 32, no. 200, pp. 675–701, 1937.

[14] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," J. Mach. Learn. Res., vol. 7, pp. 1–30, 2006.

[15] S. Mirjalili and A. Lewis, "S-shaped versus V-shaped transfer functions for binary particle swarm optimization," Swarm Evol. Comput., vol. 9, pp. 1–14, 2013.

[16] J. Kennedy and R. Eberhart, "Particle swarm optimization," in Proc. IEEE Int. Conf. Neural Netw. (ICNN), Perth, Australia, 1995, pp. 1942–1948.

[17] J. Kennedy and R. C. Eberhart, "A discrete binary version of the particle swarm algorithm," in Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC), Orlando, FL, USA, 1997, pp. 4104–4108.

[18] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: optimization by a colony of cooperating agents," IEEE Trans. Syst., Man, Cybern. B, vol. 26, no. 1, pp. 29–41, 1996.

[19] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl. (CISDA), Ottawa, Canada, 2009, pp. 1–6.

[20] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in Proc. Int. Conf. Inf. Syst. Secur. Privacy (ICISSP), 2018, pp. 108–116.

[21] R. Kohavi and G. H. John, "Wrappers for feature subset selection," Artif. Intell., vol. 97, no. 1–2, pp. 273–324, 1997.

[22] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," J. Mach. Learn. Res., vol. 3, pp. 1157–1182, 2003.

[23] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002.